COMP 469
Fall 2026

Take-Home Discussion 2

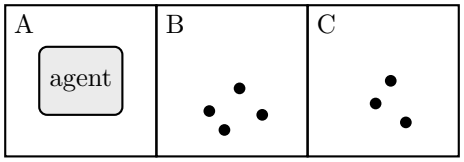
Chapter 2: Intelligent Agents Name(s): _____

Covers AIMA 4e §2.1–§2.4.7. **No search algorithm is needed anywhere in this handout.** If you find yourself describing how to *compute* a route, you have wandered into the next chapter — every question here is about what an agent perceives, what counts as success, what kind of environment it faces, and which agent design that environment demands. Work in groups of two or three. Answers are graded on reasoning, not length.

INSTRUCTOR EDITION. Solutions shown in green italics.

1 The Three-Square Vacuum World

Extend the vacuum world of Figure 2.2 to three squares in a row, A, B, and C. Each square is either Clean or Dirty. The agent perceives its own location and the status of the square it occupies, so a percept looks like $[B, \textit{Dirty}]$. The available actions are *Left*, *Right*, *Suck*, and *NoOp*. The agent starts in A. Moving into a wall leaves the agent where it is. Sucking cleans the current square, and clean squares stay clean.



Unless a part says otherwise, the performance measure awards one point for each clean square at each time step, over a lifetime of 1000 steps.

(a) How many distinct percepts are there? How many distinct percept sequences of length exactly t? Write a closed-form expression for the number of rows a lookup table needs in order to cover every percept sequence of length 1 through T.

Solution. *There are $3 \times 2 = 6$ percepts, so 6^t sequences of length exactly t, and*

$$\sum_{t=1}^{T} 6^t = \frac{6^{T+1} - 6}{5}$$

rows in total.

(b) Tabulate the agent function for every percept sequence of length one. Fill in an action that is rational under the performance measure above.

Percept sequence	Action
[*A*, *Clean*]	*Right*
[*A*, *Dirty*]	*Suck*
[*B*, *Clean*]	*Right*
[*B*, *Dirty*]	*Suck*
[*C*, *Clean*]	*Left*
[*C*, *Dirty*]	*Suck*

Solution. *Suck whenever the current square is dirty; otherwise move toward a square the agent has not just cleaned. At B, either Left or Right is defensible — movement is free and the agent has no evidence favoring one side — as long as the choice is stated. At C, only Left makes progress, since Right runs into the wall.*

(c) Write a simple reflex agent program, in at most six lines of pseudocode, that produces exactly the table you filled in for part (b). Then evaluate your expression from part (a) at $T = 10$ and compare the two numbers. State in one sentence what the comparison is meant to show.

Solution.

```
function REFLEX-VACUUM([location, status]) returns an action
  if status = Dirty then return Suck
  else if location = C then return Left
  else return Right
```

At $T = 10$ the table needs $(6^{11} - 6)/5 = 72{,}559{,}410$ rows, roughly 7.3×10^7, against four lines of program. The table is an external description of the behavior; the program is what we actually build, and it is small because it exploits a regularity in the environment — the right action for a dirty square is the same no matter what came before.

(d) Now change the performance measure so that each movement costs one point, while *Suck* and *NoOp* are free. Is your agent from part (c) still rational? If not, say what behavior becomes rational once the floor is entirely clean, and name the action that makes it possible.

Solution. *No. Once all three squares are clean the agent keeps shuttling between them and loses a point per step for the rest of its 1000-step lifetime. The rational behavior is to stop, which requires NoOp. Nothing about the program changed — only the performance measure did. Rationality is a relation among the performance measure, the prior knowledge, the action set, and the percept sequence, not a property of the code.*

(e) Your part (c) agent cannot actually tell when the floor is clean, because its action depends only on the current percept. What is the smallest capability you must add, what is the name Chapter 2 gives the resulting design, and which two models does that design maintain?

Solution. *Add internal state: a record of which squares have been observed clean since they were last visited (plus the agent's own location if it must be tracked). That is a **model-based reflex agent**. It maintains a **transition model** — how the world changes on its own and in response to the agent's actions — and a **sensor model** — what the percepts say about the world state.*

(f) Suppose the location sensor fails, so the percept is now just *Clean* or *Dirty*. Explain why no simple reflex agent with a deterministic rule set can be rational here. What partial fix does the chapter offer, and why does it call it only a trick rather than a solution?

Solution. *The percept Clean now occurs at A, B, and C, and those three situations call for different actions, but a deterministic reflex agent must return the same action for the same percept. A rule that always moves Right on Clean gets stuck against the wall at C; one that always moves Left gets stuck at A. Randomizing the choice will eventually escape the loop and, in a world this small, do reasonably well. The chapter treats randomization as a useful trick rather than a general answer: it escapes loops without supplying any knowledge, and in larger or safety-critical tasks random action is inefficient or dangerous. The real fix is internal state.*

2 PEAS and Two Task Environments

Consider two systems.

- **Agent I** — a tournament chess program playing under a game clock.
- **Agent II** — an autonomous hospital delivery robot that carries supplies between wards along corridors it shares with people, gurneys, and cleaning carts.

(a) Write a PEAS description for Agent II with at least two concrete items in each category. "Be helpful" and "do a good job" are not performance measures.

Performance	*Deliveries completed on time; zero collisions with people or equipment; correct item delivered to the correct ward; battery managed so the robot is never stranded; low corridor obstruction time.*
Environment	*Corridors, elevators, doorways, ward layouts, pedestrians and staff, gurneys and carts, spills and closed doors, delivery requests, shift schedules.*
Actuators	*Drive wheels, steering or differential drive, brakes, cargo compartment latch, elevator call signal, warning tone and status display.*
Sensors	*Lidar or depth camera, bump and proximity sensors, wheel odometry, floor map and localization, cargo-bay sensor, battery gauge, network feed of delivery requests.*

(b) Classify both agents on the seven dimensions of §2.3.2. One or two words per cell.

Dimension	**I: chess with a clock**	**II: hospital robot**
Observable	*Fully*	*Partially*
Agents	*Multiagent, competitive*	*Multiagent, mostly cooperative*
Determinism	*Deterministic*	*Nondeterministic*
Episodes	*Sequential*	*Sequential*
Time	*Semidynamic*	*Dynamic*
State / action	*Discrete*	*Continuous*
Knowledge	*Known*	*Partly unknown*

Solution. *Chess is fully observable and known: both players see the whole board and the rules are given in advance. It is deterministic — a legal move has exactly one outcome — even though the opponent's* choice *is uncertain, which is a multiagent property, not a determinism property. Getting this distinction right is the point of the row.*

(c) Students most often miss the **Time** row. Explain why chess with a clock is semidynamic rather than static, and name a task environment from §2.3.2 that is simply static.

Solution. *The board does not change while a player deliberates, so the environment itself is static; but the clock runs, so the agent's performance score changes with deliberation time. That combination is exactly what* semidynamic *means. A crossword puzzle is static: neither the world nor the score moves while you think.*

(d) *Known* and *observable* are independent. For the hospital robot, describe (i) a situation in which its environment is known but only partially observable, and (ii) a change that would make some part of it unknown while it remains fully observable.

Solution. *(i) The floor plan, the rules of motion, and the elevator's behavior are all known in advance, yet the robot cannot see a gurney parked around the next corner — known, partially observable. (ii) Install a new automated door whose signalling protocol the robot has never used. A ceiling camera may show the robot the entire corridor, so the state is fully observable, but the effect of its own "open door" action is not yet known — fully observable, unknown. Known/unknown is about the designer's and agent's grip on the* rules*, not about what the sensors can see.*

(e) The chapter separates *nondeterministic* from *stochastic*. Which label applies to the robot's model of a pedestrian stepping into its path, and what would have to change for the other label to apply?

Solution. *As stated, nondeterministic: the model lists possible outcomes — the pedestrian steps left, steps right, or stops — without quantifying them. It becomes stochastic only when the model attaches probabilities, for example a 15% chance of stepping into the path. Both mean the next state is not determined by the current state and action; only stochastic models commit to numbers.*

(f) A vendor proposes the performance measure "minimize average delivery time." Give one concrete way a perfectly rational agent could maximize this measure while producing behavior the hospital would reject. Rewrite the measure to close the loophole, and name the problem from Chapter 1 that this illustrates.

Solution. *Any answer that optimizes the stated number rather than the intended outcome. Examples: the robot serves only nearby wards and lets distant requests sit in the queue forever, so the average over* completed *deliveries looks excellent; or it drives fast through crowded corridors, trading safety for speed the measure never scores. A repaired measure scores the state of the world we actually want: on-time completion rate across* all *requests including the slowest, zero collisions and near-misses, correct item and ward, and a cap on worst-case wait rather than an average. This is the King Midas problem, or value alignment — a capable optimizer faithfully maximizing a badly specified objective. Note also that an average conceals variance: two robots with the same average delivery time can behave very differently, and if variance matters the measure has to say so.*

3 Agent Designs, Rationality, and Representation

(a) For each system below, select the **least complex** agent design from Chapter 2 that is adequate for the task as described. Choose exactly one bubble per row.

(i) A smoke alarm that sounds whenever particulate density crosses a fixed threshold.
• simple reflex ◯ model-based reflex ◯ goal-based ◯ utility-based ◯ learning

(ii) A floor-cleaning robot whose sensors cannot tell which room it is in, but which must cover every room.
◯ simple reflex • model-based reflex ◯ goal-based ◯ utility-based ◯ learning

(iii) A delivery drone that must reach a specified address; any route that arrives is equally acceptable.
◯ simple reflex ◯ model-based reflex • goal-based ◯ utility-based ◯ learning

(iv) A router that must weigh a faster arrival against a higher chance of missing a pickup window.
◯ simple reflex ◯ model-based reflex ◯ goal-based • utility-based ◯ learning

(v) A mail filter deployed at an organization whose spam patterns were not known at design time.
◯ simple reflex ◯ model-based reflex ◯ goal-based ◯ utility-based • learning

Solution. *(i) The correct action is a function of the current percept alone. (ii) Coverage requires remembering where it has been, and the percept does not supply that — partial observability calls for internal state. (iii) A binary goal test is enough because all successful outcomes are equally good. (iv) Two outcomes both satisfy the goal but differ in quality and carry uncertainty, which is exactly what a utility function is for. (v) The relevant regularities are unknown at design time, so the agent has to improve its own components from feedback. Note that (v) is not exclusive with the others: a learning agent is built* on top of *one of the first four designs.*

(b) True or false, with a one-line justification for each.

(i) An agent that chose a rational action and then suffered the worst possible outcome acted irrationally. **F**

(ii) A rational agent never has reason to take an action whose only payoff is information. **F**

(iii) Whether an agent is rational can be determined by inspecting its program. **F**

(iv) An agent that relies entirely on its designer's prior knowledge lacks autonomy. **T**

Solution. *(i) False. Rationality maximizes* expected *performance given the evidence available at the time; perfection would maximize actual performance, and demanding it would require a crystal ball. (ii) False. Information gathering is rational whenever better percepts improve later decisions — looking both ways before crossing does not move you across the street, and is still the right action. (iii) False. Rationality is a relation among the performance measure, prior knowledge, action set, and percept sequence; the same four lines of code are rational under one specification and irrational under another. (iv) True. That is what the dung beetle and the sphex wasp illustrate: behavior that is rational only in the exact environment their built-in assumptions describe, and that degrades to nonsense the moment the assumption is violated.*

(c) A table-driven agent stores one entry per percept sequence. Let $|P|$ be the number of distinct percepts and T the agent's lifetime in steps. Write the number of table entries, then give the **three independent** reasons the chapter says this design fails, and name the analogy it offers for what should replace the table.

Solution. *The table needs* $\sum_{t=1}^{T} |P|^t$ *entries. The three failures are independent: no physical agent has room to* store *the table; no designer has time to* build *it; and no agent could ever have enough experience to* learn *all the right entries. The design is not incorrect — filled in properly it implements exactly the agent function we want — it is simply unbuildable. The analogy is the replacement of large printed square-root tables by Newton's method: a short program that captures the structure which generated the table. The chapter's wager is that the same can be done for intelligent behavior in general.*

(d) Match each learning-agent component to its job, then answer the follow-up.

Performance element	*Selects the external action — everything we previously called "the agent."*
Critic	*Judges how the agent is doing against a fixed external performance standard.*
Learning element	*Uses the critic's feedback to modify the performance element's components.*
Problem generator	*Proposes actions that look suboptimal now but produce informative experience.*

Follow-up: why must the performance standard be fixed and sit conceptually *outside* the agent?

Solution. *Percepts carry no verdict on their own — a chess program can perceive that it has delivered checkmate and still need a standard to know that this is good news. If the agent could edit that standard, it could always succeed by lowering the bar: redefining success instead of improving behavior. This is the slide to point back to when reward hacking comes up later in the course.*

(e) Return to the three-square vacuum of Question 1. Write the state "the agent is in B, A is clean, B is dirty, C is clean" in each of the three representations.

Solution. ***Atomic:*** *one indivisible label such as* s_{17}*, with no internal parts the agent can inspect.* ***Factored:*** *a fixed set of variables with values, e.g.* $loc = B$*,* $dirtA = false$*,* $dirtB = true$*,*

dirtC = false. ***Structured:*** *objects and relations, e.g. At(agent, B), Dirty(B), RightOf(B, A), RightOf(C, B).*

(f) Which of the three representations is required to express "every square to the left of the agent is clean" without enumerating the cases one by one, and what does the chapter say you pay for that expressiveness?

Solution. *Structured. The claim quantifies over objects and depends on a relation between them, which neither an atomic label nor a fixed list of variables can state in general — a factored representation can only enumerate the specific combinations. The price is that reasoning becomes computationally harder as the representation becomes more expressive; the chapter presents atomic → factored → structured as an axis of increasing expressiveness with increasing cost.*

(g) Is a one-hot encoding of the agent's location localist or distributed? Give one advantage of the alternative.

Solution. *Localist: one dimension stands for one concept, so exactly one component is on. The distributed alternative spreads the meaning across many components, which makes it more robust to noise — a small perturbation moves the representation to a nearby point rather than flipping it to an unrelated symbol — and lets similar states end up near each other.*

4 Pac-Man as a Chapter 2 Agent

In the two lecture demonstrations, the same simple reflex Pac-Man program was run on an easy corridor level and then on a level with branches and detours. It did well on the first and poorly on the second.

(a) In Chapter 2 vocabulary, what changed between the two runs and what did not? What does this tell you about judging an architecture from one successful run?

Solution. *The agent program was unchanged: the same ordered condition–action rules mapping the current percept to a move, with no memory, no model of the maze, no goal, and no utility. Only the task environment changed — the second level makes the task genuinely sequential, so the locally convenient action stops being the globally useful one. Success in the first level was a property of the environment, not evidence about the architecture: a rule set can look adequate whenever the task effectively reduces to local reactions and fail as soon as it does not.*

(b) A classmate says "it failed because its path-finding routine was too slow." Using the equation *agent = architecture + program*, explain in two or three sentences why this diagnosis is wrong.

Solution. *There is no path-finding routine to blame — the program never computes a route at all. The architecture supplies the sensors, actuators, and compute; the program is the mapping from percepts to actions, and this program's mapping consults only the current percept. The failure is not speed but information: the decision procedure has no representation of remaining food elsewhere in the maze and no target for a route to end at, so no amount of extra speed would change its output.*

(c) Write a performance measure for Pac-Man that rewards survival and nothing else. Describe the degenerate behavior a perfectly rational agent would produce under it, then repair the measure.

Solution. *For example, "one point per time step in which Pac-Man is alive." A rational agent under this measure retreats to the safest corner of the maze and stays there: it never clears a pellet, never finishes the level, and scores extremely well. The measure rewards the wrong state of the environment. A repair scores the outcome actually wanted — pellets cleared and level completion, with survival as a constraint or a penalty term rather than the whole objective — for instance points per pellet, a bonus for clearing the maze, and a large penalty for being caught. Same lesson as Question 2(f) in a different costume.*